

# Arduino и конечные автоматы

## чистая организация кода

После многих лет отладки запутанных скетчей для Arduino, которые выглядели как «спагетти-код», я усвоил, что конечные автоматы — это не просто теория из информатики. Это основа для написания поддерживаемого встраиваемого ПО. Как инженер по печатным платам, который отлаживал бесчисленные проблемы прошивки в 2 часа ночи, могу сказать: правильная реализация конечного автомата — это разница между проектом, который доходит до релиза, и тем, который нет.

## Основы конечных автоматов в Arduino

Конечный автомат в Arduino — это паттерн программирования, при котором система в любой момент времени находится в одном из нескольких определённых состояний. Автомат переходит между состояниями на основе конкретных условий или событий. Представьте светофор: он всегда находится ровно в одном состоянии (красный, жёлтый или зелёный) и переключается между ними по заранее заданным правилам.

Красота конечных автоматов становится очевидной, когда вы управляете сложным поведением. Вместо вложенных `if-else`, которые невозможно отследить, вы организуете код в дискретные состояния с чёткой логикой переходов.

## Почему конечные автоматы важны для встраиваемых систем

В реальных проектах печатных плат с микроконтроллерами вы постоянно жонглируете несколькими параллельными операциями: чтение датчиков, управление исполнительными механизмами, работа с протоколами связи и обработка пользовательского ввода. Без структуры это быстро становится неуправляемым.

Конечные автоматы обеспечивают:

- **Предсказуемое поведение:** вы всегда знаете, в каком состоянии находится система
- **Упрощённую отладку:** проблемы локализуются в конкретных состояниях
- **Поддерживаемый код:** можно добавлять или изменять состояния, не ломая существующую логику
- **Хорошую документацию:** диаграммы состояний служат исполняемой документацией
- **Меньше ошибок:** исключаются состояния гонки и неожиданные комбинации состояний

## Автоматы Мура и Мили

В разработке встраиваемых систем вы встретите два основных типа конечных автоматов.

## Характеристики автомата Мура

| Особенность              | Автомат Мура                                    |
|--------------------------|-------------------------------------------------|
| Определение выхода       | Только по текущему состоянию                    |
| Стабильность выхода      | Более стабильный, меняется только при переходах |
| Типичное число состояний | Обычно требует больше состояний                 |
| Лучший сценарий          | Когда критична стабильность выхода              |
| Сложность реализации     | Проще в реализации и понимании                  |

В автомате Мура выходы определяются исключительно тем, в каком состоянии вы находитесь. Светодиод либо включён, либо выключен в зависимости от состояния, а не от того, как вы в него попали.

## Характеристики автомата Мили

| Особенность              | Автомат Мили                                 |
|--------------------------|----------------------------------------------|
| Определение выхода       | По текущему состоянию И входу                |
| Стабильность выхода      | Может меняться во время выполнения состояния |
| Типичное число состояний | Обычно требует меньше состояний              |
| Лучший сценарий          | Когда нужна немедленная реакция на входы     |
| Сложность реализации     | Более сложная логика переходов               |

Автоматы Мили формируют выходы во время переходов между состояниями. Они более отзывчивы, но их сложнее реализовать корректно.

## Практический пример сравнения

Рассмотрим систему управления двигателем. В автомате Мура у вас будут состояния `MOTOR_FORWARD`, `MOTOR_REVERSE`, `MOTOR_STOPPED`. Действие двигателя определяется самим состоянием. В автомате Мили у вас может быть меньше состояний, но действие двигателя зависит и от текущего состояния, и от сигнала датчика, вызвавшего переход.

Для большинства проектов на Arduino я рекомендую начинать с автоматов Мура — их проще понимать и отлаживать.

## Реализация конечного автомата в Arduino с помощью `enum` и `switch/case`

Самый чистый способ реализовать конечный автомат в Arduino — использовать перечисления ( `enum` ) в сочетании с `switch/case` . Этот подход даёт читаемый код и эффективное выполнение.

## Базовая структура: определение `enum`

```
// Define all possible states
enum SystemState {
    STATE_INIT,
    STATE_IDLE,
    STATE_RUNNING,
    STATE_PAUSED,
    STATE_ERROR,
    STATE_SHUTDOWN
};
// Create state variable
SystemState currentState = STATE_INIT;
```

Перечисления дают человекочитаемые имена вместо «магических чисел». Компилятор воспринимает их как ваш собственный тип, обеспечивая типобезопасность, при этом под капотом используются эффективные целочисленные значения.

## Полный шаблон конечного автомата

```
enum State {
    IDLE,
    ACTIVE,
    ERROR
};
State currentState = IDLE;
unsigned long stateStartTime = 0;
void setup() {
    Serial.begin(9600);
    // Initialize hardware
    pinMode(LED_BUILTIN, OUTPUT);
}
void loop() {
    // State machine execution
    switch(currentState) {
        case IDLE:
            handleIdleState();
            break;
        case ACTIVE:
            handleActiveState();
            break;
        case ERROR:
```

```

        handleErrorState();
        break;
    default:
        // Defensive programming - handle unexpected state
        Serial.println("ERROR: Unknown state!");
        currentState = IDLE;
        break;
    }
}

void handleIdleState() {
    digitalWrite(LED_BUILTIN, LOW);
    // Transition logic
    if (Serial.available()) {
        currentState = ACTIVE;
        stateStartTime = millis();
        Serial.println("Transitioning to ACTIVE");
    }
}

void handleActiveState() {
    digitalWrite(LED_BUILTIN, HIGH);
    // Timeout after 5 seconds
    if (millis() - stateStartTime > 5000) {
        currentState = IDLE;
        Serial.println("Timeout - returning to IDLE");
    }
}

void handleErrorState() {
    // Blink LED to indicate error
    digitalWrite(LED_BUILTIN, (millis() / 250) % 2);
    // Reset on button press
    if (digitalRead(RESET_BUTTON) == LOW) {
        currentState = IDLE;
        Serial.println("Error cleared");
    }
}
}

```

## Реальный пример: контроллер последовательности светодиодов

Покажу практический пример, демонстрирующий правильный дизайн конечного автомата — многопаттерновый контроллер светодиодов.

### Дизайн конечного автомата

```

enum LEDPattern {
    PATTERN_OFF,
    PATTERN_SOLID,

```

```

    PATTERN_SLOW_BLINK,
    PATTERN_FAST_BLINK,
    PATTERN_PULSE
};
LEDPattern currentPattern = PATTERN_OFF;
unsigned long lastTransitionTime = 0;
int ledBrightness = 0;
bool ledDirection = true;
const int LED_PIN = 9; // PWM capable pin
void setup() {
    pinMode(LED_PIN, OUTPUT);
    Serial.begin(9600);
}
void loop() {
    unsigned long currentTime = millis();
    switch(currentPattern) {
        case PATTERN_OFF:
            analogWrite(LED_PIN, 0);
            checkPatternButton();
            break;
        case PATTERN_SOLID:
            analogWrite(LED_PIN, 255);
            checkPatternButton();
            break;
        case PATTERN_SLOW_BLINK:
            if (currentTime - lastTransitionTime >= 1000) {
                digitalWrite(LED_PIN, !digitalRead(LED_PIN));
                lastTransitionTime = currentTime;
            }
            checkPatternButton();
            break;
        case PATTERN_FAST_BLINK:
            if (currentTime - lastTransitionTime >= 200) {
                digitalWrite(LED_PIN, !digitalRead(LED_PIN));
                lastTransitionTime = currentTime;
            }
            checkPatternButton();
            break;
        case PATTERN_PULSE:
            // Smooth fading effect
            analogWrite(LED_PIN, ledBrightness);
            if (ledDirection) {
                ledBrightness += 5;
                if (ledBrightness >= 255) {
                    ledBrightness = 255;
                    ledDirection = false;
                }
            } else {
                ledBrightness -= 5;
                if (ledBrightness <= 0) {

```

```

        ledBrightness = 0;
        ledDirection = true;
    }
}
checkPatternButton();
delay(30);
break;
}
}
void checkPatternButton() {
    if (buttonPressed()) {
        currentPattern = (LEDPattern)((currentPattern + 1) % 5);
        lastTransitionTime = millis();
        Serial.print("Pattern changed to: ");
        Serial.println(currentPattern);
    }
}
}

```

## Продвинутые паттерны конечных автоматов

### Действия при входе и выходе из состояния

Профессиональные встраиваемые системы часто нуждаются в коде, который выполняется один раз при входе в состояние или выходе из него. Вот надёжный паттерн:

```

enum State {
    STATE_A,
    STATE_B,
    STATE_C
};
State currentState = STATE_A;
State previousState = STATE_A;
void loop() {
    // Detect state changes
    if (currentState != previousState) {
        exitState(previousState);
        enterState(currentState);
        previousState = currentState;
    }
    // Execute state logic
    executeState(currentState);
}
void enterState(State state) {
    switch(state) {
        case STATE_A:
            Serial.println("Entering State A");

```

```

        digitalWrite(LED_PIN, HIGH);
        break;
    case STATE_B:
        Serial.println("Entering State B");
        startMotor();
        break;
    case STATE_C:
        Serial.println("Entering State C");
        resetCounters();
        break;
    }
}

void exitState(State state) {
    switch(state) {
        case STATE_A:
            Serial.println("Exiting State A");
            break;
        case STATE_B:
            Serial.println("Exiting State B");
            stopMotor();
            break;
        case STATE_C:
            Serial.println("Exiting State C");
            saveData();
            break;
    }
}

void executeState(State state) {
    switch(state) {
        case STATE_A:
            // State A continuous logic
            break;
        case STATE_B:
            // State B continuous logic
            break;
        case STATE_C:
            // State C continuous logic
            break;
    }
}

```

## Иерархические конечные автоматы

Для сложных систем можно реализовать вложенные состояния:

```

enum MainState {
    SYSTEM_OFF,
    SYSTEM_OPERATING,

```

```

SYSTEM_FAULT
};
enum OperatingSubState {
    OP_STARTUP,
    OP_NORMAL,
    OP_CALIBRATION,
    OP_SHUTDOWN
};
MainState mainState = SYSTEM_OFF;
OperatingSubState opState = OP_STARTUP;
void loop() {
    switch(mainState) {
        case SYSTEM_OFF:
            handleSystemOff();
            break;
        case SYSTEM_OPERATING:
            // Handle sub-state machine
            switch(opState) {
                case OP_STARTUP:
                    handleStartup();
                    break;
                case OP_NORMAL:
                    handleNormalOperation();
                    break;
                case OP_CALIBRATION:
                    handleCalibration();
                    break;
                case OP_SHUTDOWN:
                    handleShutdown();
                    break;
            }
            break;
        case SYSTEM_FAULT:
            handleFault();
            break;
    }
}

```

## Тайминг и неблокирующий код в конечных автоматах

Один из критически важных аспектов встраиваемых конечных автоматов — правильный тайминг без блокировок. **Никогда не используйте `delay()` в production-автоматах.**

### Сравнение методов тайминга

| Метод                 | Блокирующий | Точность | Сложность    | Лучше всего для      |
|-----------------------|-------------|----------|--------------|----------------------|
| <code>delay()</code>  | Да          | Хорошая  | Очень низкая | Только простые демо  |
| <code>millis()</code> | Нет         | Хорошая  | Низкая       | Большинство проектов |
| <code>micros()</code> | Нет         | Отличная | Низкая       | Точный тайминг       |
| Прерывания таймера    | Нет         | Отличная | Высокая      | Критичный тайминг    |

## Реализация неблокирующего тайминга

```
enum TimedState {
    WAIT_START,
    WAIT_COMPLETE,
    PROCESSING
};
TimedState state = WAIT_START;
unsigned long stateTimer = 0;
const unsigned long WAIT_DURATION = 3000; // 3 seconds
void loop() {
    unsigned long currentMillis = millis();
    switch(state) {
        case WAIT_START:
            digitalWrite(LED_PIN, LOW);
            Serial.println("Starting wait period...");
            stateTimer = currentMillis;
            state = WAIT_COMPLETE;
            break;
        case WAIT_COMPLETE:
            // Non-blocking wait
            if (currentMillis - stateTimer >= WAIT_DURATION) {
                state = PROCESSING;
                Serial.println("Wait complete, processing...");
            }
            // Do other work while waiting
            checkSensors();
            break;
        case PROCESSING:
            digitalWrite(LED_PIN, HIGH);
            performWork();
            state = WAIT_START; // Loop back
            break;
    }
}
```

# Типичные ошибки при работе с конечными автоматами и как их избежать

## Ошибка 1: Отсутствие default-ветки

**Проблема:** неопределённое поведение при повреждении переменной состояния.

**Решение:** всегда включайте `default`, обрабатывающий неожиданные состояния:

```
switch(currentState) {
  case STATE_A:
    // Handle state A
    break;
  case STATE_B:
    // Handle state B
    break;
  default:
    Serial.println("ERROR: Invalid state detected!");
    currentState = STATE_INIT; // Safe fallback
    logError();
    break;
}
```

## Ошибка 2: Забытые break

**Проблема:** код «проваливается» в следующий case.

**Решение:** всегда используйте `break` или явно комментируйте намеренный fall-through:

```
switch(state) {
  case CALIBRATING:
    startCalibration();
    break; // CRITICAL: Don't fall through
  case RUNNING:
    // Intentional fall-through
    // (uncommon but sometimes useful)
  case MONITORING:
    readSensors();
    break;
}
```

## Ошибка 3: Блокирующие операции в состояниях

**Проблема:** использование `delay()` не даёт другим состояниям выполняться.

**Решение:** используйте `millis()` для тайминга и разбивайте работу на части:

```

// BAD - Blocks for 5 seconds
void badStateHandler() {
    doWork();
    delay(5000); // Terrible!
    doMoreWork();
}
// GOOD - Non-blocking
unsigned long workStartTime = 0;
bool workInProgress = false;
void goodStateHandler() {
    if (!workInProgress) {
        doWork();
        workStartTime = millis();
        workInProgress = true;
    } else if (millis() - workStartTime >= 5000) {
        doMoreWork();
        workInProgress = false;
    }
}
}

```

## Ошибка 4: Переходы состояний внутри обработчиков

**Проблема:** несколько изменений состояния за одну итерацию цикла ведут к непредсказуемому поведению.

**Решение:** используйте переменные-флаги и выполняйте переход после обработки состояния:

```

// Better approach
State nextState = currentState;
void handleState() {
    switch(currentState) {
        case STATE_A:
            if (condition) {
                nextState = STATE_B; // Set next state
                return; // Don't continue processing
            }
            // Continue STATE_A logic
            break;
    }
    // Apply transition after all state logic
    if (nextState != currentState) {
        currentState = nextState;
    }
}
}

```

# Отладка конечных автоматов

## Логирование истории состояний

```
const int HISTORY_SIZE = 10;
State stateHistory[HISTORY_SIZE];
int historyIndex = 0;
void logStateChange(State newState) {
    stateHistory[historyIndex] = newState;
    historyIndex = (historyIndex + 1) % HISTORY_SIZE;
    Serial.print("State transition: ");
    Serial.print(currentState);
    Serial.print(" -> ");
    Serial.println(newState);
}
void printStateHistory() {
    Serial.println("Recent state history:");
    for (int i = 0; i < HISTORY_SIZE; i++) {
        Serial.print(stateHistory[i]);
        Serial.print(" ");
    }
    Serial.println();
}
```

## Визуализация конечного автомата

Создайте функцию для вывода статуса автомата в целях отладки:

```
void printStateMachineStatus() {
    Serial.println("=== State Machine Status ===");
    Serial.print("Current State: ");
    Serial.println(getStateName(currentState));
    Serial.print("Time in State: ");
    Serial.print(millis() - stateStartTime);
    Serial.println(" ms");
    Serial.print("Transitions Today: ");
    Serial.println(transitionCount);
    Serial.println("=====");
}
const char* getStateName(State s) {
    switch(s) {
        case IDLE: return "IDLE";
        case ACTIVE: return "ACTIVE";
        case ERROR: return "ERROR";
        default: return "UNKNOWN";
    }
}
```

```
}  
}
```

# Лучшие практики проектирования конечных автоматов

## Принципы для production-кода

1. **Держите состояния сфокусированными:** каждое состояние должно иметь одну чёткую цель
2. **Минимизируйте число состояний:** слишком много состояний — признак плохого дизайна
3. **Документируйте переходы:** комментируйте, почему происходит каждый переход
4. **Используйте осмысленные имена:** `STATE_WAITING_FOR_SENSOR_STABILIZATION` лучше, чем `STATE_3`
5. **Планируйте ошибки:** всегда включайте состояния ошибок и пути восстановления
6. **Тестируйте переходы:** проверяйте каждый возможный путь перехода
7. **Избегайте глубокой вложенности:** если нужны вложенные автоматы, подумайте о рефакторинге

## Таблица переходов состояний

| Текущее состояние | Вход/Условие   | Следующее состояние | Действие                                    |
|-------------------|----------------|---------------------|---------------------------------------------|
| IDLE              | Нажатие кнопки | ACTIVE              | Запустить двигатель, записать метку времени |
| ACTIVE            | Тайм-аут (5 с) | IDLE                | Остановить двигатель                        |
| ACTIVE            | Ошибка датчика | ERROR               | Остановить двигатель, записать ошибку       |
| ERROR             | Кнопка сброса  | IDLE                | Сбросить флаг ошибки                        |
| ЛЮБОЕ             | Аварийный стоп | EMERGENCY           | Немедленное отключение                      |

## Библиотека ресурсов для разработки конечных автоматов на Arduino

### Официальная документация

- **Arduino Reference:** полный справочник по языку на [arduino.cc](https://www.arduino.cc/reference/)
- **State Machine Library:** доступна через Arduino Library Manager
- **Arduino Forum:** активное сообщество на [forum.arduino.cc](https://forum.arduino.cc/)

# Инструменты разработки

## Инструменты диаграмм состояний:

- [Draw.io](#) (бесплатный веб-инструмент для диаграмм)
- PlantUML (генерация диаграмм из текста)
- Lucidchart (профессиональное построение диаграмм)

## Анализ кода:

- Arduino IDE Serial Monitor
- Visual Studio Code с расширением Arduino
- PlatformIO для продвинутой отладки

## Библиотеки и фреймворки

- **StateMachine Library**: <https://github.com/jrullan/StateMachine>
- **Arduino-FSM**: лёгкая реализация FSM
- **Automaton**: событийно-ориентированный фреймворк конечных автоматов

## Учебные ресурсы

- «Design Patterns for Embedded Systems in C» Брюса Пауэла Дугласа
- Спецификации UML State Machine: документация стандартов OMG
- Embedded Systems Programming: статьи о дизайне FSM
- Учебники по конечным автоматам на Arduino: обучающие ресурсы SparkFun и Adafruit

## Репозиторий примеров проектов

- **GitHub Arduino Examples**: поиск по «arduino state machine examples»
- **Instructables**: проекты с конечными автоматами от сообщества
- [Hackster.io](#): профессиональные встраиваемые проекты с конечными автоматами

## Инструменты тестирования аппаратуры

- **Логические анализаторы**: для отладки переходов состояний (Saleae, DSLogic)
- **Осциллографы**: проверка требований по таймингу
- **Анализаторы последовательных протоколов**: отладка автоматов связи

## Часто задаваемые вопросы

**В1: Когда использовать конечный автомат вместо простых if-else?**

Используйте конечный автомат, когда вашему коду нужно помнить, что он делал, и принимать решения на основе этой истории. Если вы пишете вложенные `if-else`, проверяющие несколько булевых флагов, или отслеживаете «в каком я режиме», вам нужен конечный автомат. Как правило, если у системы более 3–4 различных режимов работы или поведений, конечный автомат сделает код чище и поддерживаемее. Простые одноразовые решения можно оставить как `if-else`, но последовательные операции или сложные рабочие процессы tremendously выигрывают от организации через конечный автомат.

## **B2: Как обрабатывать несколько параллельных конечных автоматов в одном скетче Arduino?**

Запускайте несколько автоматов, выполняя блок `switch/case` каждого из них последовательно в `loop()`. У каждого автомата должен быть свой тип `enum` и своя переменная состояния. Например, у вас могут быть `MotorState motorState` и `LEDState ledState`, работающие независимо. Ключ — убедиться, что каждый автомат неблокирующий, чтобы все они получали процессорное время. Если автоматам нужно взаимодействовать, используйте общие переменные или флаги событий, которые один автомат устанавливает, а другой проверяет. Избегайте прямого изменения переменных состояния других автоматов — используйте систему сообщений или событий.

## **B3: Каково влияние конечных автоматов на производительность Arduino?**

Конечные автоматы, реализованные через `switch/case`, чрезвычайно эффективны — компилятор обычно генерирует таблицу переходов, которая выполняется за константное время независимо от числа состояний. Хорошо спроектированный конечный автомат фактически улучшает производительность по сравнению с вложенными цепочками `if-else`, потому что `switch/case` оптимизируется на этапе компиляции. Значения `enum` — это просто целые числа, поэтому накладных расходов по памяти нет. Основной фактор производительности — сколько работы вы делаете внутри каждого состояния: держите обработчики состояний компактными и избегайте блокирующих операций. На Arduino Uno вы легко можете обрабатывать несколько конечных автоматов с десятками состояний каждый без заметного влияния на производительность.

## **B4: Можно ли использовать конечные автоматы с прерываниями и как обрабатывать изменения состояний из ISR?**

Да, но с осторожностью. Обработчики прерываний (ISR) должны быть как можно короче, поэтому не помещайте выполнение всего конечного автомата в ISR. Вместо этого используйте ISR для установки флага или изменения простой переменной состояния, а фактическую логику автомата обрабатывайте в основном цикле. Объявляйте любые переменные, разделяемые между ISR и основным кодом, как

`volatile` , чтобы предотвратить проблемы с оптимизацией компилятора. Распространённый паттерн — прерывание устанавливает переменную `requestedState` , а основной цикл проверяет её и выполняет фактический переход. Так ваши ISR остаются быстрыми, а поведение конечного автомата — корректным.

## **B5: Как документировать и визуализировать конечные автоматы для командной работы?**

Начните с диаграммы состояний, показывающей все состояния в виде кругов/прямоугольников и переходы в виде стрелок. Подписывайте каждый переход условием, которое его вызывает. Рекомендую использовать инструменты вроде [Draw.io](https://draw.io) или PlantUML, которые можно версионировать вместе с кодом. В коде добавьте блок комментариев в начале реализации автомата с ASCII-диаграммой состояний или ссылкой на файл диаграммы. Включите в документацию таблицу переходов состояний со всеми допустимыми переходами. Для production-кода поддерживайте эту документацию в актуальном состоянии при изменении автомата — устаревшие диаграммы хуже, чем их отсутствие. Многие команды хранят диаграммы состояний как часть технических проектных документов и ссылаются на них при code review.